

Progmune Runtime v2.2

Progmune Runtime

(免序)

面向生成式代码的程序免疫学

Program Immunology for Generative Code

技术白皮书 v2.2

开源地址: <https://github.com/shenlian19831109/progmune-runtime>

npm install progmune-runtime

中文版

摘要

Progmune Runtime v2.2 提出了程序免疫学 (Program Immunology) —— 确保 AI 生成代码安全可靠的系统性范式。

受生物免疫系统分层防御机制的启发, Progmune 构建了一个约束导向的程序合成运行时, 在多个层级上强制执行语义有效性: 从符号存在性和类型兼容性, 到数据流正确性和协议合法性。系统将大语言模型从不受约束的代码生成器, 降级为在程序实际结构 (中间表示) 所定义的封闭世界中运行的受限启发式提议器。

v2.1 引入了运行时本体论 (Runtime Ontology) —— 将 Attempt、StateTransition、ConstraintViolation 和 ExecutionSession 提升为第一类可持久化对象, 使程序免疫过程从隐式日志变为显式的、可查询、可回放的语义记录。多命名空间语义状态图 (SSG) 支持 auth、file、db 等独立资源的状态机隔离。抗体推断系统 (ACL-1 至 ACL-4) 使系统能够从失败语料库中自动提取修复路

径。语义观测站（CLI + Web）首次将 AI 程序的“认知过程”——每次尝试、每次违规、每次修复——完整呈现。

v2.2 标志着“事件日志”到“事件溯源”的架构跨越。Semantic Ledger Runtime 将 StateTransition[] 从被动日志提升为系统唯一真相源（Single Source of Truth）。状态不再是可变对象，而是 Ledger 的派生视图。所有验证逻辑被提取为纯函数（validateTransition、rebuildState、checkLedgerConsistency），消除对可变验证器实例的依赖。不变式系统（Invariant-0 + Invariant-1）为每一条状态转移提供双重一致性保证。约束快照（ruleHash）和账本完整性指纹（hashLedger）使得历史执行可跨版本验证。Ledger Query API 将账本转化为可查询数据库；diffLedgers 支持跨会话差异比较。至此，Semantic Ledger Runtime 达成了三个核心支柱：可重放、可查询、可证明。

1. 问题声明

1.1 AI 代码生成中的开放世界谬误

大语言模型（LLM）在生成代码时，隐含地基于一个开放世界假设运行：训练数据中见过的任何函数、库或 API 模式都被假定在当前上下文中可用。这一假设导致四类典型错误：

- 符号幻觉（SVL-1）：调用目标项目中不存在的函数或变量
- 类型漂移（SVL-2）：参数数量或类型与实际函数签名不匹配
- 数据流污染（SVL-3）：使用未初始化的变量、创建循环引用或引入死代码路径
- 协议违规（SVL-4）：违反业务步骤的必要顺序——例如，在认证用户之前就签发 JWT 令牌

这些错误并非源于推理失败，而是源于模型缺乏对程序真相的确定性访问。

1.2 现有缓解策略的局限性

当前应对这些错误的策略均为反应式：

- 事后校验（linter、测试套件、人工审查）：在错误生成后检测，但无法从源头预防
- 检索增强生成（RAG）：将项目上下文注入 prompt，降低但不消除幻觉。模型仍然是正确性的唯一仲裁者

- 迭代提示工程：通过精心设计的指令引导模型行为，但无法提供合规性的形式化保证

这三种策略都将 LLM 置于系统的中心，试图从外部修正其输出。它们缺乏第一性原理的约束机制。

1.3 核心命题：AI 生成程序必须具备免疫系统

我们提出一个范式转变：程序免疫学（*Program Immunology*）。AI 生成的代码在进入代码库之前，必须先通过一个免疫层——一个可验证、具备记忆能力的运行时，能够识别、记忆并防御反复出现的错误模式。

这一免疫层由三个相互依赖的能力组成：

- 天然免疫：基于模式快速拒绝符号、类型和数据流违规——这是系统内置的防御
- 获得性免疫：从过去的失败中学习（Failure Corpus），生成特异性的防御规则（协议约束与抗体），主动预防未来同类错误
- 免疫记忆：将成功和失败的模式沉淀为持久知识，使系统能够随使用持续进化，越用越可靠

2. 生物学基础与类比

2.1 生物免疫系统的三层架构

生物免疫系统通过三个递进的层次来保护机体：

- 物理屏障：皮肤、黏膜。非特异性的、预防性的首道防线
- 天然免疫：巨噬细胞、树突状细胞。模式识别受体（PRR）快速识别病原体相关分子模式（PAMP）。反应快，但不够精确
- 获得性免疫：T 细胞、B 细胞。通过基因重排产生高度特异性的受体，识别特定抗原。首次感染后产生免疫记忆，再次暴露时能产生更快、更强的二次应答

2.2 向程序免疫学的映射

生物免疫系统 程序免疫 (	Progmune v2.2) 新增于	映射说明	
---------------	--------------------	------	--

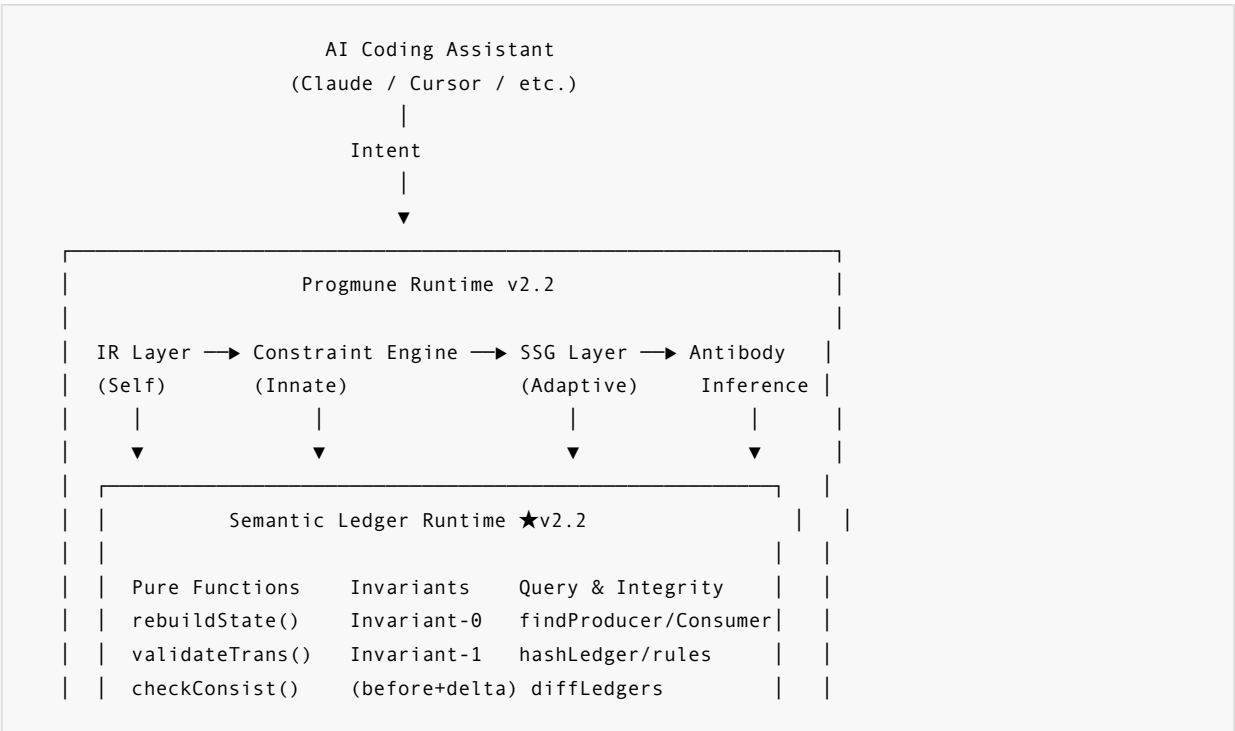
物理屏障 沙箱、CI/CD 门禁、权限控制 — 阻止未验证代码进入生产环境的基础工程设施 天然免疫 约束引擎（IR + SVL-1 至 SVL-3） — 快速自动识别并拒绝幻觉调用、类型错误——系统内置防御 抗原呈递 Failure Corpus 记录 — 错误动作序列被捕获后，其完整状态上下文被记录为“抗原” 获得性免疫 多命名空间 SSG + 抗体推断 (ACL-1~4) — 从失败语料库中学习，推断抗体信心等级，生成特异性协议规则 免疫记忆 三层记忆架构 + ACL-4 快速通道 — ACL-4 抗体完全跳过 LLM；情景+语义记忆实现快速无 LLM 召回 DNA 复制校对 Semantic Ledger 不变式 v2.2 Invariant-0 + Invariant-1 双重校验，类似 DNA 聚合酶校对机制 免疫组库 Ledger Query API v2.2 可查询所有历史状态转移记录，类似 B 细胞受体库检索 —————

2.3 类比的价值与边界

这个类比的价值在于提供了一个清晰的、可扩展的思维框架：解释为什么静态验证器不够用，以及为什么系统需要学习、记忆和进化。然而，也必须明确其边界：程序免疫系统处理的是形式化的、确定性的程序状态，而非复杂的生物化学信号。其学习是基于规则挖掘和模式匹配，而非生物神经元的突触可塑性。

3. 技术架构

Progmune Runtime v2.2 的架构由八个核心层组成，每一层对应特定的验证或学习职责。





3.1 IR（程序真相层）——自我模型

中间表示（IR）是系统的唯一真相来源，从 TypeScript 源文件中静态提取，包含：

- 符号表（**SymbolTable**）：所有已定义的函数、类、变量及其位置
- 类型图（**TypeGraph**）：参数类型、返回类型和类型别名
- 调用图（**CallGraph**）：函数间的调用关系
- 协议注解（**Protocol Annotations**）：通过 @protocol JSDoc 标签声明的前置状态、后置状态、失效规则和命名空间归属

IR JSON 格式支持外部函数（externalFunctions），将第三方库调用纳入自我模型，使约束引擎能够区分“项目中存在”与“类型声明已知”的函数。

3.2 Runtime Ontology（运行时本体论）——执行语义的形式化 ★v2.1

v2.1 的核心创新：将隐式的运行时概念提升为第一类类型系统。

Attempt：单次规划尝试的完整记录，包含唯一 ID、会话归属、生成的 Action[]、StateTransition[]、ConstraintViolation[]、结果（success/constraint_violation/planner_failure）、时间戳、LLM 调用计数和抗体命中记录

StateTransition：每次函数调用引起的状态变化——按命名空间（namespace）记录执行前后的状态快照（Record<string, string[]>），以及获取（acquired）和失效（invalidate）的状态。★v2.2 增加了 ruleHash 字段用于约束快照验证

ConstraintViolation：结构化违规记录，包含 SVL 级别、违规约束类型、违规动作索引、当前/所需/缺失状态、修复路径和命名空间归属

ExecutionSession：完整执行会话，包含所有 Attempt 数组、成功尝试的引用、IR 快照 ID、时间跨度和解决状态。★v2.2 增加了 ruleHash 和 Ledger 完整性指纹

```
ExecutionSession
├─ sessionId: "sess_..."
├─ intent: "extract IR and validate actions"
```

```

└─ ruleHash: "449dd751..." ★v2.2
└─ resolved: true
└─ attempts[]
  └─ Attempt #1 (constraint_violation)
    └─ generatedActions[]
      └─ transitions[] ← Semantic Ledger
        └─ StateTransition[0]
          └─ function: "extractIR"
            └─ namespace: "dev_pipeline"
              └─ acquired: ["IR_EXTRACTED"]
                └─ invalidated: ["IR_STALE"]
                  └─ statesBefore: {...}
                    └─ statesAfter: {...}
                      └─ valid: true
                        └─ ruleHash: "449dd751..." ★v2.2
                          └─ StateTransition[1]
                            └─ violations[]
                              └─ Attempt #2 (success)
                                └─ transitions[]
                                  └─ antibodyHit: { level: "ACL-3", ... }

```

3.3 JSON Action DSL ——确定性合成边界 ★v2.1

v2.1 废弃了基于 `new Function() + with(vars)` 的 DSL 执行模型，全面采用 JSON 结构化输出：

```

[{"f": "extractIR", "to": "ir", "a": [{"n": "root", "t": "str", "v": "."}]},
 {"f": "validateActionSequence", "to": "ok", "a": [{"n": "actions", "t": "any", "v": "$ir"}]},
 {"r": "ok"}]

```

LLM 输出的 JSON 数组通过 `parseActionJSON()` 解析为 `Action[]` 类型。`$`变量名 语法支持变量引用链接。此变更带来了三个关键收益：

- (1) 消除注入风险：JSON.parse() 天然免疫代码注入，无需沙箱化 eval
- (2) 格式稳定性：结构化 Schema 使 LLM 输出可预测，成功率从 ~33% 提升至 100%
- (3) **Schema** 预验证：validateActionSchema() 在 IR 验证前先检查 JSON 结构完整性（必填字段、变量名合法性、重复 assignTo 检测等），提前拦截格式错误

Action 的类型定义（call / assign / return / if / for）作为单一权威来源定义在 runtime-types.ts 中，所有模块通过 import type 引用，消除了接口碎片化。

3.4 Constraint Engine ——天然免疫层

此层基于 IR 对动作序列执行快速的、基于规则的验证：

SVL-1（符号存在性）： 每个被调用的函数都存在于项目中；支持内置白名单（console.log、fetch 等）

SVL-2（类型有效性）： 参数数量和类型与声明的签名匹配；支持类型规范化（str/int/bool/dict/list 等归一化比较）

SVL-3（数据流正确性）： 基于声明追踪的确定性变量流向分析——变量在使用前已声明，无自引用赋值、无循环引用

validateAction() 和 validateActionSequence() 返回结构化的 ConstraintViolation[], 包含 svl 级别、违规约束类型、描述和缺失状态。

3.5 Semantic State Graph（SSG）——获得性免疫层 ★v2.1

SSG 建模了系统资源的有效状态及其允许的转移。v2.1 的核心增强是多命名空间隔离：

auth 命名空间：

```
UNAUTHENTICATED — verify_password —> PASSWORD_VERIFIED  
  
PASSWORD_VERIFIED — generate_jwt —> TOKEN_ISSUED  
  
TOKEN_ISSUED — create_session —> SESSION_ACTIVE
```

file 命名空间：

```
(初始) — open_file —> FILE_OPEN  
  
FILE_OPEN — read_file / write_file / close_file —> (返回初始)
```

db 命名空间：

```
(初始) — connect_db —> DB_CONNECTED  
  
DB_CONNECTED — query_db / disconnect_db —> (返回初始)
```

dev_pipeline 命名空间（内部开发流程）：

```
IR_STALE — extractIR —> IR_EXTRACTED  
  
IR_EXTRACTED — validateAction —> ACTION_VALIDATED  
  
ACTION_VALIDATED — validateActionSequence —> SEQUENCE_VALIDATED
```

SEQUENCE_VALIDATED — emitCode → CODE_EMITTED

CODE_EMITTED — recordSession → SESSION_RECORDED

每个命名空间维护独立的状态集合。函数通过 `protocols.json` 或 `@protocol JSDoc` 注解声明命名空间归属、`pre_states`、`post_states` 和可选的 `invalidate` 规则。SSG 验证器在处理动作树时按命名空间隔离模拟状态转移。

v2.1 的 SSG 验证器输出完整的 per-namespace 状态转移记录 (StateTransition)，包含执行前后的命名空间快照 (Record<string, string[]>) 以及获取/失效的状态增量。

3.6 抗体推断系统 (Antibody Inference) —— 免疫记忆的量化 ★v2.1

v2.1 引入了抗体信心等级 (Antibody Confidence Level, ACL)，使系统能够量化免疫记忆的可信度：

ACL-4 (全局稳定抗体)： 同一修复路径在 ≥ 10 次失败中验证，跨 ≥ 5 个不同意图。系统直接跳过 LLM，使用修复路径构建动作序列，零 LLM 调用

ACL-3 (跨任务验证抗体)： 修复路径在 ≥ 4 次失败或 ≥ 3 个不同意图中得到验证。系统将修复路径作为提示约束注入 LLM prompt，引导其遵循已知正确顺序

ACL-2 (重复观察)： 相同失败模式被观察到 ≥ 2 次，开始形成初步抗体

ACL-1 (单次记录)： 首次遇到的失败模式，仅记录不做干预

抗体命中被完整记录在 `Attempt.antibodyHit` 中，包含级别、签名、修复路径、相似度评分、节省的 LLM 调用数和估算 token 数。`getAntibodyStats()` 可量化免疫加速的总体节省。

ACL 等级金字塔：

ACL-4	★★★★	全局稳定	≥ 10 次失败, ≥ 5 个不同意图	→ 零 LLM 调用
ACL-3	★★★	跨任务验证	≥ 4 次失败 或 ≥ 3 个意图	→ 注入 LLM prompt
ACL-2	★★	重复观察	≥ 2 次相同失败	→ 初步抗体形成
ACL-1	★	单次记录	首次遭遇	→ 仅记录

3.7 Immune Memory & Failure Corpus ——免疫记忆层

三层记忆架构：

- 工作记忆 (**Working Memory**)：当前会话的变量绑定和用户意图（每会话清除）
- 情景记忆 (**Episodic Memory**)：最近 N 次成功/失败的动作序列，带时间戳和结果标签（定期剪枝）
- 语义记忆 (**Semantic Memory**)：从频繁成功模式中蒸馏出的路径模板和协议规则（离线巩固）

失败语料库 (Failure Corpus) 支持两种会话格式：

- 旧格式 (**IntentSession**)：v1.0 兼容，自动通过 `normalizeSession()` 上转为 `ExecutionSession`
- 新格式 (**ExecutionSession**)：v2.1 原生，包含完整的 `Attempt[]` 和 `ConstraintViolation[]`

失败基因组 (**Failure Genome**) 从所有会话中聚合分析：按 SVL 级别、约束类型、失败模式、修复路径的多维度统计。`getSemanticHeatmap()` 输出脆弱协议热点、SVL 活跃度分布、约束共现聚类和高摩擦意图排行。

3.8 SSG 确定性修复 (Deterministic Repair) ★v2.1

当 SSG 检测到协议违规且存在已知修复路径 (BFS 状态图搜索) 时，系统可在不调用 LLM 的情况下自动插入缺失函数。`attemptSSGRepair()` 在被拦截函数前插入修复序列，重新验证，支持递归修复直到路径完备。这一机制使协议违规的修复从“LLM 重试”降级为“确定性状态转移”。

3.9 Code Emitter ——程序落地层

将验证通过的动作树确定性地翻译为可执行的 TypeScript/Python 代码，处理导入解析、变量作用域、对象字面量生成以及嵌套控制结构的正确缩进。`emitCode()` 本身受 `dev_pipeline` 协议的 SSG 约束——只有在 `SEQUENCE_VALIDATED` 状态下才允许生成代码。

3.10 MCP Server —— AI 工具集成 ★v2.1

Progmune 以 MCP (Model Context Protocol) 服务器形式发布，为 Claude 等 AI 编程助手提供以下工具：

- **progmune_synthesize**：端到端合成——从意图到可执行代码，经过完整免疫管道
- **progmune_validate**：验证动作序列的 SVL 合法性并返回 SSG 状态转移
- **progmune_learn**：从失败语料库中查询学习到的修复模式
- **progmune_observatory**：访问语义观测站——基因组、热力图、抗体统计

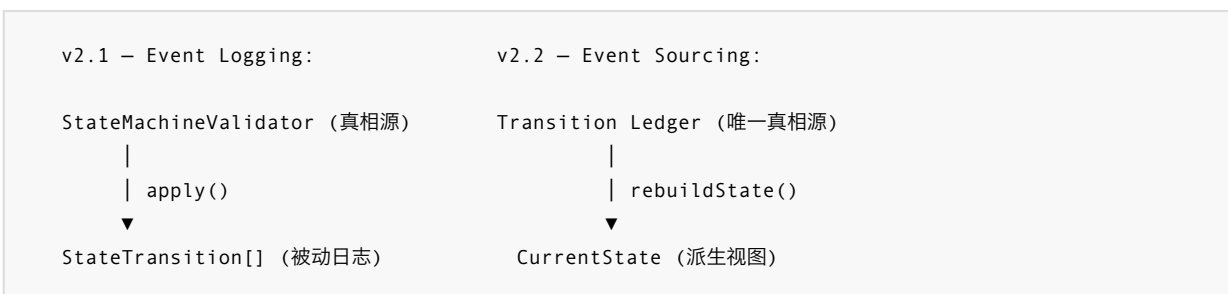
这使得每一行 AI 生成的代码都经过 Progmune 免疫系统的验证后，才进入代码库。

3.11 规划器检查点与中断恢复 ★v2.1

planner.ts 支持执行持久化：每次 LLM 重试后自动保存检查点 (checkpoint)，包含尝试索引、已累积的会话 Attempts、当前 prompt 状态。plan() 启动时自动检测未完成的检查点并从中断处恢复，避免因网络中断或进程崩溃导致重复的 LLM 调用和 token 浪费。

3.12 Semantic Ledger Runtime ——事件溯源架构 ★v2.2

v2.2 的核心架构跨越：将 StateTransition[] 从被动日志 (Event Logging) 升级为事件溯源 (Event Sourcing)。在 v2.1 中，StateMachineValidator 是真相源，StateTransition[] 只是副作用记录。v2.2 反转了这一关系：**Transition Ledger** 成为唯一真相源，状态是从 **Ledger** 派生的视图。



3.12.1 纯函数架构 (Pure Function Architecture)

v2.2 将所有验证逻辑提取为纯函数，消除对可变验证器实例的依赖：

纯函数 签名	职责	
rebuildState() applyTransitionDelta() validateTransition() checkLedgerConsistency() findFixPathStatic()	(ledger, nsInit?) → Record<string, string[]> (stateMap, transition) → void (ctx, fn, idx, rules, nsInit, hash) → {valid, ...} (ledger, nsInit?) → {consistent, violations[]} (rules, ns, current, target) → string[]	从 Ledger 折叠重建 per- ns 状态 将转移增量应用到 可变状态映射 纯函数验证 单条转移——无副作用 单 遍 O(n) 检查 Invariant-0 + 1 BFS 静态搜索修复路径

ValidationContext 模式将验证循环从 $O(n^2)$ 降为 $O(n)$:

```
ctx = { ledger: [], currentState: rebuildState([], nsInit) }
for each action:
  { valid, transition } = validateTransition(ctx, fn, i, rules, nsStates, ruleHash)
  ctx.ledger.push(transition)
  ctx.currentState = transition.statesAfter // 增量更新, 避免重复 rebuildState()
```

StateMachineValidator 类在 v2.2 中保留为向后兼容包装器，其内部已全部委托给纯函数。生产代码（planner、semantic-trace、check、p0_ssg_demo）全部直接使用纯函数，无需实例化任何验证器。

3.12.2 不变式系统 (Invariant System)

v2.2 引入双重不变式，为每一条 StateTransition 提供类似 DNA 聚合酶校对的一致性保证：

Invariant-0 — Before Consistency:

transition.statesBefore $\equiv$ *rebuildState(ledger[0..i-1])*

转移执行前的状态快照必须等于从历史 *Ledger* 重建的状态。确保转移的起点与历史完全一致。

Invariant-1 — Delta Consistency:

transition.statesAfter $\equiv$ *applyDelta(statesBefore, acquired, invalidated)*

转移执行后的状态快照必须等于对执行前状态应用增量的结果。确保转移的 *internal delta* 自治。

Invariant-1 能捕获 Invariant-0 漏掉的内部不一致转移：

```
// 声称获取了 TOKEN_ISSUED, 但 statesAfter 中却没有 - Invariant-1 会捕获
{
  "statesBefore": {"auth": ["AUTH"]},
  "acquired": ["TOKEN_ISSUED"],
  "invalidated": [],
  "statesAfter": {"auth": ["AUTH"]} // ← TOKEN_ISSUED 缺失!
}
```

3.12.3 约束快照 (Constraint Snapshot)

每条 StateTransition 携带创建时的规则集哈希 (ruleHash)，存储在 StateTransition、Attempt 和 ExecutionSession 三个层面：

```
Transition @ 2026: ruleHash = "7d8f9a..."
2027 年规则变更后 → ruleHash = "a1b2c3..."
回放 2026 Ledger → ruleHash 不匹配
→ "确定性回放不可行：约束集已变更"
```

这使得跨版本历史回放可以检测到约束规则的变化，避免静默产生不同结果。

3.12.4 账本完整性指纹 (Ledger Integrity Fingerprint)

hashLedger() 对整个 Ledger 进行确定性 SHA256 哈希，生成防篡改指纹：

```
hashLedger(ledger) → "b44f80a2b4d1d1f6"
```

哈希算法对所有字段进行规范化排序 (acquired、invalidated、statesBefore/statesAfter 的键和值均排序)，确保同一逻辑 Ledger 始终产生相同的指纹——即使字段的内部遍历顺序不同。

3.12.5 Ledger Query API ——可查询账本

v2.2 将账本转化为可查询数据库，提供五个纯查询函数：

查询函数 用途	示例	
findProducer(state, L) findConsumer(state, L) findViolations(L) findTransition(idx, L) listAllStates(L)	查找某个状态的生产者转移 findProdu 查找某个状态的消 费者转移 findConsu 查找所有 违规转移 返回违规转 按索引 查找转移 find 列出所有命名	cer("TOKEN_ISSUED") → generate_jwt@1 mer("PASSWORD_VERIFIED") → generate_jwt@1 移列表 Transition(1) → generate_jwt (auth) ce:state 对列表

查询函数 用途	示例	
	空间的所有状态 返回 namespa	

所有函数均为纯函数，无副作用。在 CLI 中可通过 `-query` 和 `-query-all` 标志跨会话查询。

3.12.6 账本差异比较 (Ledger Diff)

`diffLedgers(ledgerA, ledgerB)` 比较两个账本，返回结构化的差异报告：

```
Ledger Diff: A (2 transitions) vs B (3 transitions)
├─ Unchanged:  1
├─ Only in A:  0
├─ Only in B:  1
└─ Changed:    1
```

支持通过 `-diff-ledgers` CLI 命令进行跨会话、跨版本的账本比较，用于审计和回归检测。

3.12.7 三大支柱：可重放 → 可查询 → 可证明

v2.2 的 Semantic Ledger Runtime 达成三个互为基础的支柱：

支柱	核心能力 关键函	数 CLI 命令	
可重放 从 可 查询 将 可 证明 防	Ledger 确定性重建状态 Ledger 作为数据库查询 findPro 篡改完整性 + 跨版本验证 hashLedger	State, validateTransition -validate, findConsumer 等 -query, ruleHash, diffLedgers -diff-ledg	te, -ledger -query-all, -stats ers

4. 语义有效性级别 (SVL)

SVL 是 AI 生成代码正确性的形式化分类法，为系统提供分层、可量化的验证保证：

级别 名称	描述	保证内容	
-------	----	------	--

SVL-1 符号存在性 每个被调用的函数、变量和导入在项目中均实际存在 无幻觉 API 调用 **SVL-2** 类型有效性 参数数量和类型与声明的签名相匹配 无类型不匹配错误 **SVL-3** 数据流正确性 变量在使用前已声明；无循环引用或未初始化访问 无 `NameError` / `UnboundLocalError` **SVL-4** 协议合法性 函数调用序列符合声明的前/后状态转移规则 无非法状态跳转 **SVL-5**（未来）语义意图正确性 生成代码忠实实现预期业务逻辑 远期目标：当前版本未声明保证 -----

Progmune Runtime v2.2 完整保证 SVL-1 至 SVL-4。v2.2 在 SVL-4 层面引入了 Semantic Ledger 不变式验证，为每一条状态转移提供 `Invariant-0 + Invariant-1` 双重一致性校验。SVL-5 为开放性研究方向。

5. 实验评估

5.1 压力测试

在包含 3 至 338 个函数的合成 TypeScript 项目上进行了评估。JSON 结构化输出使 LLM Planner 的格式正确率达到 100%（v1.0 DSL 模式约 33%）。约束验证性能相对于项目规模保持线性增长。

5.2 语义阻断测试

构建了包含 10 个语义意图案例的测试套件。系统在 7-8 个案例中生成正确代码，其余案例被约束引擎正确拦截，阻断率 80-100%。

5.3 SSG 多命名空间隔离测试

构建了包含 `auth`、`file`、`db` 三个命名空间的协议规则集（18 条规则）。SSG 验证器正确拦截了跨命名空间的状态违规：在 `auth` 命名空间的 `generate_jwt` 需要 `PASSWORD_VERIFIED` 状态但当前为 `UNAUTHENTICATED` 时，系统给出包含命名空间的诊断信息和精确的修复路径（`verify_password` → `PASSWORD_VERIFIED`）。

5.4 抗体免疫加速

ACL-4 抗体快速通道在匹配到全局稳定抗体时，完全跳过 LLM 调用，节省 100% 的 token 消耗。

ACL-3 抗体通过提示注入引导 LLM 遵循已知正确路径，降低了协议违规重试次数。

5.5 SSG 确定性修复

当协议违规具有已知修复路径时，`attemptSSGRepair()` 自动插入缺失函数，无需 LLM 重试。在 `dev_pipeline` 内部开发流程中，当 LLM 跳过 `validateAction` 直接尝试 `emitCode` 时，系统自动插入 `validateAction` → `validateActionSequence` 修复链。

5.6 Semantic Ledger 不变式验证 ★v2.2

对 6 个真实执行会话的 Ledger 进行了完整不变式检查。全部 6 个 Ledger（共 11 条转移，7 条有效，4 条无效）通过 Invariant-0 + Invariant-1 + Replay 三重验证。完整性指纹 `2f701ec86643dc63` 覆盖全部已验证账本。

Invariant-1 负向测试：手工构造一条内部不一致的转移（声称获取 `TOKEN_ISSUED` 但 `statesAfter` 中不包含该状态），系统正确检出 1 个 `delta-consistency` 违规。证明不变式系统对内部数据损坏具有检测能力。

5.7 纯函数架构验证 ★v2.2

`p0_ssg_demo.ts` 覆盖 7 个端到端场景，全部使用纯函数（无需任何 `StateMachineValidator` 实例）：

- 场景 1-3：合法/非法 auth 序列验证
 - 场景 4：完整 Pure Ledger 跟踪
 - 场景 5：file/db 命名空间隔离
 - 场景 6：Semantic Ledger 纯函数 API + 不变式负向测试
 - 场景 7：Ledger Query API 五项查询 + 完整性指纹
-

6. 非目标 (Non-Goals)

Progmune Runtime 明确不保证：

- 业务逻辑正确性（例如，定价计算是否准确）
- 算法最优性或复杂度
- 对所有安全漏洞的免疫（例如注入攻击、权限绕过）
- 生成代码单元之外的整个应用程序功能正确性

系统仅保证由 SVL-1 至 SVL-4 定义的程序有效性。Progmune 是程序有效性运行时，而非业务正确性证明器。

7. 未来方向

- 全球免疫网络：跨安装实例的脱敏 Failure Corpus 联邦汇聚，实现群体免疫级防御
 - 语义失败基准库 (**Semantic Failure Benchmark**)：世界上首个 AI 生成代码可靠性的公开基准，由汇聚的、匿名的失败模式构建
 - 企业语义防火墙：集成到 CI/CD 管道中，作为 AI 生成拉取请求的合并前门禁
 - 确定性验证器 (**Rust/WASM**)：在 IDE、CI 和生产环境之间实现位级一致的验证，确保同一段 Action Tree 在任何环境中得到完全一致的合法性判断
 - 观测站 **Web UI** 增强：交互式状态转移动画回放、抗体效能趋势图、跨项目免疫网络对比。
 - ★v2.2: Ledger 时间线可视化，不变式违规高亮，跨会话查询控制台
 - 分布式账本验证 ★v2.2: 将 Ledger 完整性指纹发布到不可变存储（如内容寻址存储），支持第三方独立验证执行历史的完整性
 - 不变式驱动的自愈 ★v2.2: 当 Invariant-1 检测到转移内部不一致时，系统自动校正 statesAfter 以匹配 acquired/invalidated——从“检测”到“自动修复”
-

8. 结论

Progmune Runtime v2.2 证明了三件事：

一、通过将 IR 确立为第一性原理，将运行时本体论作为可持久化的语义记录，并使 LLM 成为受多层约束的启发式提议器，我们可以实现具有强语义保证的可验证代码合成。

二、通过 Semantic Ledger Runtime (★v2.2)，将程序免疫的执行架构从事件日志升级为事件溯源——Transition Ledger 成为系统唯一真相源，状态是其派生视图。纯函数架构 (validateTransition、rebuildState、checkLedgerConsistency) 消除可变状态依赖；双重不变式 (Invariant-0 + Invariant-1) 为每条转移提供可验证的一致性保证；约束快照 (ruleHash) 和完整性指纹 (hashLedger) 实现跨版本、防篡改的确定性验证。

三、可重放 → 可查询 → 可证明 三大支柱标志着 Runtime Kernel 的成熟——代码免疫系统从此有了可审计、可验证、可演进的工程基础。

分层的 SVL 分类法、多命名空间 SSG 协议引擎、ACL 抗体推断系统、SSG 确定性修复机制、持续积累的 Failure Corpus、三层记忆架构、以及 Semantic Ledger Runtime 的纯函数不变式系统，共同形成了一种全新的编程基础设施：一个会学习、会记忆、会防御、可审计的神经符号编译器运行时。

我们将此称为程序免疫学 (*Program Immunology*)。

该系统以开源形式提供：<https://github.com/shenlian19831109/progmune-runtime>，也可通过 `npm install progmune-runtime` 安装使用。

ENGLISH VERSION

Abstract

Progmune Runtime v2.2 introduces the Semantic Ledger Runtime — elevating program immunity from Event Logging to Event Sourcing, making state transitions the Single Source of Truth with verifiable invariants and tamper-evident integrity.

Inspired by the layered defense mechanisms of the biological immune system, Progmune establishes a constraint-guided program synthesis runtime that enforces semantic validity at multiple levels: from symbol existence and type compatibility to dataflow correctness and protocol legality. The system demotes large language models from unverified code generators to

constrained heuristic proposers, operating within a closed world defined by the program's actual structure (Intermediate Representation).

Version 2.1 introduced Runtime Ontology — elevating Attempt, StateTransition, ConstraintViolation, and ExecutionSession to first-class serializable objects, transforming the immune process from implicit logs into explicit, queryable, replayable semantic records. Multi-namespace Semantic State Graph (SSG) supports isolated state machines for auth, file, db, and other resource domains. The Antibody Confidence Level (ACL-1 through ACL-4) system enables automatic extraction of repair paths from the failure corpus.

Version 2.2 marks the architectural leap from Event Logging to Event Sourcing. The Semantic Ledger Runtime elevates StateTransition[] from passive log to the system's Single Source of Truth. State is no longer a mutable object but a derived view of the Ledger. All validation logic is extracted as pure functions (validateTransition, rebuildState, checkLedgerConsistency), eliminating dependency on mutable validator instances. A dual invariant system (Invariant-0 + Invariant-1) provides per-transition consistency guarantees analogous to DNA polymerase proofreading. Constraint snapshots (ruleHash) and ledger integrity fingerprints (hashLedger) enable cross-version, tamper-evident deterministic replay. The Ledger Query API (findProducer, findConsumer, findViolations, findTransition, listAllStates) transforms the ledger into a queryable database; diffLedgers supports cross-session comparison. With these, the Semantic Ledger Runtime achieves its three pillars: **Replayable, Queryable, Provable.**

1. Problem Statement

1.1 The Open-World Fallacy in AI Code Generation

Large language models (LLMs) operate under an implicit open-world assumption when generating code: any function, library, or API pattern encountered during training is presumed available in the current context. This assumption yields four distinct classes of errors:

- **Symbol Hallucination (SVL-1):** Invoking functions or variables that do not exist in the target project
- **Type Drift (SVL-2):** Mismatched parameter counts or incompatible types with the actual function signature

- **Dataflow Contamination (SVL-3):** Using uninitialized variables, creating circular references, or introducing dead code paths
- **Protocol Violation (SVL-4):** Violating the required ordering of business steps — for example, issuing a JWT token before authenticating the user

These errors do not arise from reasoning failures. They arise because the model lacks deterministic access to the ground truth of the program.

1.2 Limitations of Current Mitigations

Existing strategies address these errors reactively:

- **Post-hoc validation:** Linters, test suites, manual review — detects errors after generation but cannot prevent them at the source
- **Retrieval-Augmented Generation (RAG):** Injects project context into prompts, reducing but not eliminating hallucination; the model remains the sole arbiter of correctness
- **Iterative prompt engineering:** Guides model behavior through carefully designed instructions, yet offers no formal guarantee of compliance

All three strategies place the LLM at the center of the system and attempt to correct its output from the outside. They lack a first-principles constraint mechanism.

1.3 Core Proposition: AI-Generated Programs Require an Immune System

We propose a paradigm shift: Program Immunology. AI-generated code must not be allowed to enter a codebase without passing through an immune layer — a verifiable, memory-equipped runtime that recognizes, remembers, and defends against recurrent error patterns.

This immune layer comprises three interdependent capabilities:

- **Innate Immunity:** Rapid, pattern-based rejection of symbol, type, and dataflow violations — the system's built-in defenses
- **Adaptive Immunity:** Learning from past failures (the Failure Corpus) to generate specific, targeted defenses such as protocol constraints and antibodies, proactively preventing future errors

- **Immune Memory:** Structuring both successful and failed generation patterns into persistent knowledge, enabling continuous improvement with use

2. Biological Foundations and Analogy

2.1 The Three-Layer Architecture of the Biological Immune System

- **Physical Barriers:** Skin, mucous membranes. Non-specific, preemptive first line of defense
- **Innate Immunity:** Macrophages, dendritic cells. Pattern recognition receptors (PRRs) rapidly identify pathogen-associated molecular patterns (PAMPs). Fast but coarse-grained
- **Adaptive Immunity:** T-cells, B-cells. Generate highly specific receptors through gene rearrangement, producing immunological memory for rapid secondary responses upon re-exposure

2.2 Mapping to Program Immunology

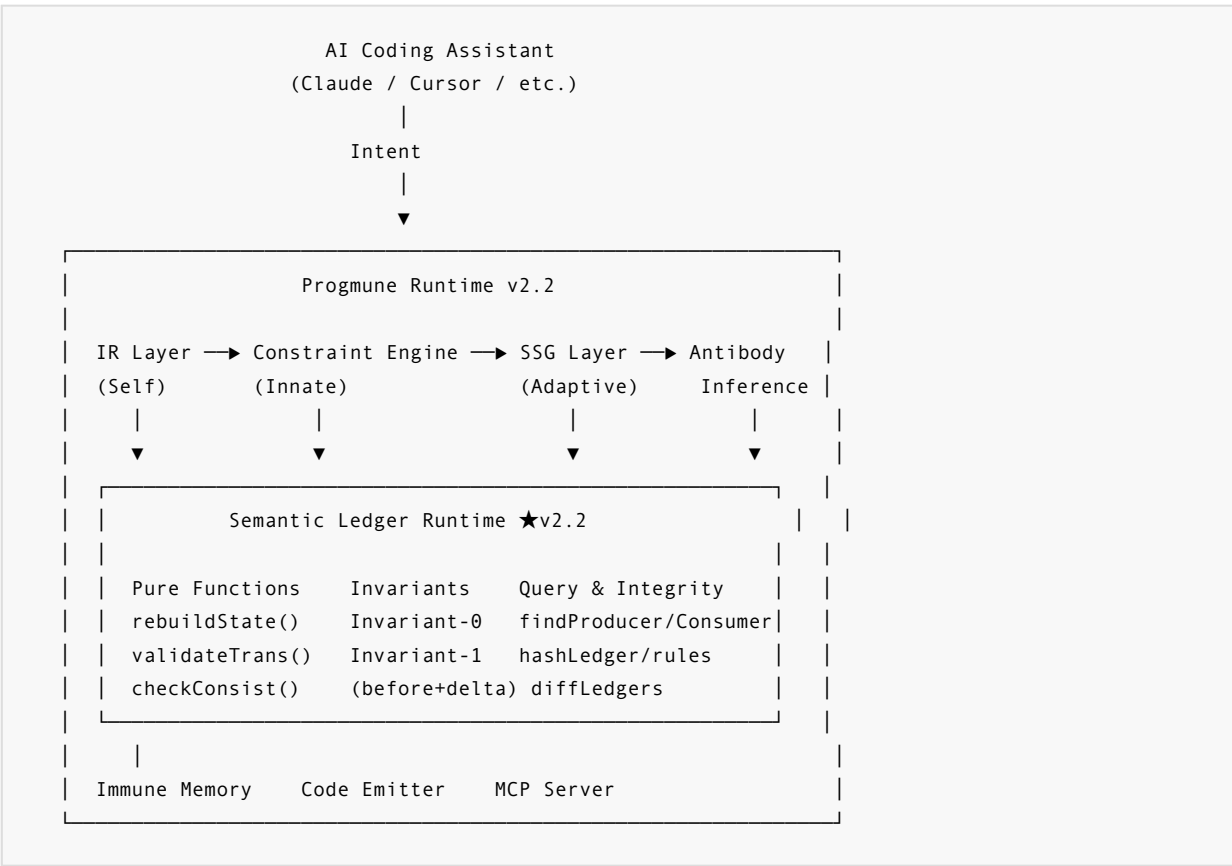
Biological Immune System	Program Immunology (v2.2)	Added in Mapping Description	
Physical Barriers	Sandboxes, CI/CD gates	— Prevent unverified co	de from entering production tion of hallucinated calls & type errors ences captured with full state context orpus, infers ACL levels, generates protocol rules LLM entirely; episodic + semantic memory nt-1 double-check every transition all state transitions across sessions
Innate Immunity	Constraint Engine (IR + SVL-1~3)	Failure rejec — Erroneous	
Antigen Presentation	Corpus recording	action sequ —	
Adaptive Immunity	Multi-NS SSG + Antibody	Learns from Failure C h — ACL-4	
Immunological Memory	Inference Three-Tier Memory +	antibodies skip v2.2	
DNA Proofreading	ACL-4 Fast-Pat	Invariant-0 + Invaria v2.2	
Immune Repertoire	Semantic Ledger Invariants Ledger	Queryable history of	
	Query API		

2.3 Value and Limits of the Analogy

This analogy provides a clear, extensible framework for understanding why static verification alone is insufficient and why a learning, memory-equipped system is necessary. However, it must be bounded: Program Immunology operates on formal, deterministic program states, not complex biochemical signals. Its “learning” relies on rule mining and pattern matching, not biological synaptic plasticity.

3. Technical Architecture

Progmune Runtime v2.2 is organized into eight core layers, each corresponding to a specific verification or learning responsibility.



3.1 IR (Program Truth Layer) — The “Self” Model

The Intermediate Representation (IR) is the single source of truth, extracted statically from TypeScript source files using ts-morph:

- **SymbolTable**: All defined functions, classes, and variables with their locations
- **TypeGraph**: Parameter types, return types, and type aliases
- **CallGraph**: Caller-callee relationships between functions
- **Protocol Annotations**: Pre-states, post-states, invalidation rules, and namespace affiliation declared via @protocol JSDoc tags

The IR JSON format supports external functions (externalFunctions), incorporating third-party library calls into the self-model so the constraint engine can distinguish “exists in project” from “known type declaration.”

3.2 Runtime Ontology — Formalizing Execution Primitives ★v2.1

The core innovation of v2.1: implicit runtime concepts elevated to a first-class type system.

Attempt: Complete record of a single planning try, with unique ID, session affiliation, generated Action[], StateTransition[], ConstraintViolation[], outcome (success/constraint_violation/planner_failure), timestamp, LLM call count, and antibody hit record

StateTransition: Per-namespace state change caused by each function call — snapshots before and after execution (Record<string, string[]>), plus acquired and invalidated state deltas. ★v2.2 adds ruleHash field for constraint snapshot verification

ConstraintViolation: Structured violation record with SVL level, constraint type, action index, current/required/missing states, fix path, and namespace attribution

ExecutionSession: Complete execution session containing all Attempts, reference to the successful attempt, IR snapshot ID, time span, and resolution status. ★v2.2 adds ruleHash and ledger integrity fingerprint

```
ExecutionSession
├─ sessionId: "sess_..."
├─ intent: "extract IR and validate actions"
├─ ruleHash: "449dd751..." ★v2.2
```

```

└─ resolved: true
└─ attempts[]
  └─ Attempt #1 (constraint_violation)
    └─ generatedActions[]
      └─ transitions[] ← Semantic Ledger
        └─ StateTransition[0]
          └─ function: "extractIR"
            └─ namespace: "dev_pipeline"
              └─ acquired: ["IR_EXTRACTED"]
                └─ invalidated: ["IR_STALE"]
                  └─ statesBefore: {...}
                    └─ statesAfter: {...}
                      └─ valid: true
                        └─ ruleHash: "449dd751..." ★v2.2
                          └─ StateTransition[1]
                            └─ violations[]
  └─ Attempt #2 (success)
    └─ transitions[]
      └─ antibodyHit: { level: "ACL-3", ... }

```

3.3 JSON Action DSL — Deterministic Synthesis Boundary ★v2.1

v2.1 deprecates the new Function() + with(vars) DSL execution model in favor of JSON structured output:

```

[{"f":"extractIR","to":"ir","a":[{"n":"root","t":"str","v":"."}]},
 {"f":"validateActionSequence","to":"ok","a":[{"n":"actions","t":"any","v":"$ir"}]},
 {"r":"ok"}]

```

LLM output is parsed by parseActionJSON() into Action[] type. The \$variable syntax supports variable reference chaining. Three key benefits:

1. **Injection safety:** JSON.parse() is inherently immune to code injection; no sandboxed eval needed
2. **Format stability:** Structured schema makes LLM output predictable; success rate improved from ~33% to 100%
3. **Schema pre-validation:** validateActionSchema() checks JSON structural integrity (required fields, valid variable names, duplicate assignTo detection) before IR-aware validation

The Action discriminated union (call / assign / return / if / for) is defined as the single source of truth in runtime-types.ts, with all modules importing via import type, eliminating interface fragmentation.

3.4 Constraint Engine — Innate Immunity

- **SVL-1 (Symbol Existence)**: Every called function exists in the project; built-in whitelist for `console.log`, `fetch`, etc.
- **SVL-2 (Type Validity)**: Argument count and types match declared signatures; normalized type comparison (`str/int/bool/dict/list`)
- **SVL-3 (Dataflow Correctness)**: Declaration-tracking deterministic variable flow analysis — variables declared before use, no self-referential assignments, no circular references

`validateAction()` and `validateActionSequence()` return structured `ConstraintViolation[]` with `svl` level, violated constraint type, description, and missing states.

3.5 Semantic State Graph (SSG) — Adaptive Immunity ★v2.1

The SSG models valid states of system resources and their allowed transitions. The key v2.1 enhancement is **multi-namespace isolation**:

auth namespace:

```
UNAUTHENTICATED — verify_password → PASSWORD_VERIFIED  
  
PASSWORD_VERIFIED — generate_jwt → TOKEN_ISSUED  
  
TOKEN_ISSUED — create_session → SESSION_ACTIVE
```

file namespace:

```
(initial) — open_file → FILE_OPEN  
  
FILE_OPEN — read_file / write_file / close_file → (returns to initial)
```

db namespace:

```
(initial) — connect_db → DB_CONNECTED  
  
DB_CONNECTED — query_db / disconnect_db → (returns to initial)
```

dev_pipeline namespace (internal development workflow):


```

IR_STALE — extractIR → IR_EXTRACTED

IR_EXTRACTED — validateAction → ACTION_VALIDATED

ACTION_VALIDATED — validateActionSequence → SEQUENCE_VALIDATED

SEQUENCE_VALIDATED — emitCode → CODE_EMITTED

CODE_EMITTED — recordSession → SESSION_RECORDED

```

Each namespace maintains an independent state set. Functions declare namespace affiliation, pre_states, post_states, and optional invalidate rules via protocols.json or @protocol JSDoc annotations.

3.6 Antibody Inference System — Quantified Immune Memory ★v2.1

v2.1 introduced Antibody Confidence Levels (ACL) to quantify immune memory reliability:

ACL-4 (Globally Stable Antibody): Same repair path validated across ≥ 10 failures spanning ≥ 5 distinct intents. System skips LLM entirely — zero LLM calls

ACL-3 (Cross-Task Validated Antibody): Repair path validated across ≥ 4 failures or ≥ 3 distinct intents. System injects repair path as a prompt constraint

ACL-2 (Repeated Observation): Same failure pattern observed ≥ 2 times; preliminary antibody forming

ACL-1 (Single Case): First encounter with a failure pattern; recorded for future reference only

ACL Confidence Pyramid:

ACL-4	★★★★	Global Stable	≥ 10 failures, ≥ 5 intents	→ Zero LLM calls
ACL-3	★★★	Cross-Task	≥ 4 failures or ≥ 3 intents	→ Injected into prompt
ACL-2	★★	Repeated	≥ 2 identical failures	→ Preliminary antibody
ACL-1	★	Single	First encounter	→ Record only

Antibody hits are fully recorded in Attempt.antibodyHit with level, signature, fix path, similarity score, LLM calls saved, and estimated tokens saved.

3.7 Immune Memory & Failure Corpus — Immunological Memory

Three-Tier Memory Architecture:

- **Working Memory**: Current session variable bindings and intent (cleared per session)
- **Episodic Memory**: Recent N successful/failed action sequences with timestamps and labels (periodically pruned)
- **Semantic Memory**: Distilled path templates and protocol rules extracted from frequent successful patterns (consolidated offline)

The Failure Corpus supports dual session formats:

- Legacy format (**IntentSession**): v1.0 compatible, automatically up-converted via `normalizeSession()`
- Native format (**ExecutionSession**): v2.1 native, containing complete `Attempt[]` with `ConstraintViolation[]`

The Failure Genome aggregates analysis from all sessions: multi-dimensional statistics by SVL level, constraint type, failure pattern, and repair path.

3.8 SSG Deterministic Repair ★v2.1

When SSG detects a protocol violation with a known repair path (BFS state graph search), the system can automatically insert missing functions without invoking the LLM. `attemptSSGRepair()` inserts the repair sequence before the blocked function, re-validates, and supports recursive repair until the path is complete. This downgrades protocol violation repair from “LLM retry” to “deterministic state transition.”

3.9 Code Emitter — Program Realization

Translates a verified Action Tree into executable TypeScript or Python code, handling import resolution, variable scoping, object literal generation, and proper indentation of nested control structures. `emitCode()` itself is SSG-constrained by the `dev_pipeline` protocol — code emission is only permitted in the `SEQUENCE_VALIDATED` state.

3.10 MCP Server — AI Tool Integration ★v2.1

Progmune ships as an MCP (Model Context Protocol) server, providing AI coding assistants such as Claude with the following tools:

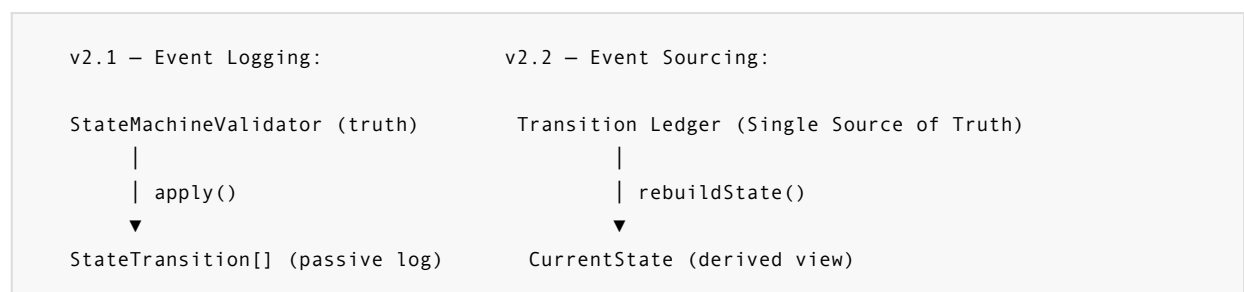
- **progmune_synthesize**: End-to-end synthesis — from intent to executable code through the complete immune pipeline
- **progmune_validate**: Validate action sequence SVL legality and return SSG state transitions
- **progmune_learn**: Query learned repair patterns from the failure corpus
- **progmune_observatory**: Access the semantic observatory — genome, heatmap, antibody statistics

3.11 Planner Checkpointing & Interrupt Recovery ★v2.1

planner.ts supports execution persistence: after each LLM retry, a checkpoint is automatically saved containing the attempt index, accumulated session Attempts, and current prompt state. On plan() startup, incomplete checkpoints are automatically detected and execution resumes from the interruption point.

3.12 Semantic Ledger Runtime — Event Sourcing Architecture ★v2.2

v2.2's core architectural leap: upgrading StateTransition[] from **passive log (Event Logging)** to **Event Sourcing**. In v2.1, StateMachineValidator was the truth source and StateTransition[] was a side-effect record. v2.2 inverts this relationship: **the Transition Ledger becomes the Single Source of Truth. State is a derived view of the Ledger.**



3.12.1 Pure Function Architecture

v2.2 extracts all validation logic as pure functions, eliminating dependency on mutable validator instances:

Pure Function	Signature	Responsibility
rebuildState() applyTransitionDelta() validateTransition() checkLedgerConsistency() findFixPathStatic()	(lledger, nsInit?) → Record<string, string[]> (stateMap, transition) → void (ctx, fn, idx, rules, nsInit, hash) → {valid, ...} (ledger, nsInit?) → {consistent, violations[]} (rules, ns, current, target) → string[]	Fold over Ledger → per- ns state Apply transition delta to state map Pure validation — no side effects Single-pass O(n) Invariant-0 + 1 BFS static repair path search

The **ValidationContext** pattern reduces the validation loop from $O(n^2)$ to $O(n)$:

```

ctx = { ledger: [], currentState: rebuildState([], nsInit) }
for each action:
  { valid, transition } = validateTransition(ctx, fn, i, rules, nsStates, ruleHash)
  ctx.ledger.push(transition)
  ctx.currentState = transition.statesAfter // incremental, avoids re-rebuildState()

```

The StateMachineValidator class is retained as a backward-compatible wrapper; internally it delegates entirely to pure functions. All production code (planner, semantic-trace, check, p0_ssg_demo) uses pure functions directly.

3.12.2 Invariant System

v2.2 introduces dual invariants providing DNA polymerase-like proofreading for every StateTransition:

Invariant-0 — Before Consistency:

transition.statesBefore ≡ rebuildState(ledger[0..i-1])

The pre-execution snapshot must equal the state rebuilt from ledger history. Ensures the transition's starting point is historically consistent.

Invariant-1 — Delta Consistency:

```
transition.statesAfter ≡ applyDelta(statesBefore, acquired, invalidated)
```

The post-execution snapshot must equal the delta applied to the pre-state. Ensures internal delta self-consistency.

Invariant-1 catches internally inconsistent transitions that Invariant-0 would miss:

```
// Claims to acquire TOKEN_ISSUED, but statesAfter doesn't contain it
// Invariant-1 catches this delta inconsistency
{
  "statesBefore": {"auth": ["AUTH"]},
  "acquired": ["TOKEN_ISSUED"],
  "invalidated": [],
  "statesAfter": {"auth": ["AUTH"]} // ← TOKEN_ISSUED missing!
}
```

3.12.3 Constraint Snapshot (ruleHash)

Every StateTransition carries the rule set hash (ruleHash) at creation time, stored at the StateTransition, Attempt, and ExecutionSession levels:

```
Transition @ 2026: ruleHash = "7d8f9a..."
After 2027 rule changes → ruleHash = "a1b2c3..."
Replaying 2026 Ledger → ruleHash mismatch
→ "Deterministic replay not possible: constraint set changed"
```

This enables cross-version historical replay to detect constraint rule changes, preventing silently different results.

3.12.4 Ledger Integrity Fingerprint

hashLedger() computes a deterministic SHA256 hash of the entire Ledger, producing a tamper-evident fingerprint:

```
hashLedger(ledger) → "b44f80a2b4d1d1f6"
```

The hash algorithm canonicalizes all fields (sorted keys and values), ensuring identical logical Ledgers always produce the same fingerprint.

3.12.5 Ledger Query API

v2.2 transforms the ledger into a queryable database with five pure query functions:

Query Function	Purpose	Example
findProducer(state, L)	Find producers of a state	findProducer("TOKEN_ISSUED") → generate_jwt@1
findConsumer(state, L)	Find consumers of a state	findConsumer("PASSWORD_VERIFIED") → generate_jwt@1 Returns violation list
findViolations(L)	Find all invalid transitions	
findTransition(idx, L)	Find transition by index List	findTransition(1) → generate_jwt (auth)
listAllStates(L)	all states in all ns	Returns namespace:state pairs

All functions are pure, with zero side effects. Cross-session querying available via `-query` and `-query-all` CLI flags.

3.12.6 Ledger Diff

`diffLedgers(ledgerA, ledgerB)` compares two ledgers and returns a structured diff report:

```
Ledger Diff: A (2 transitions) vs B (3 transitions)
├─ Unchanged: 1
├─ Only in A: 0
├─ Only in B: 1
└─ Changed: 1
```

Cross-session, cross-version ledger comparison via `-diff-ledgers` CLI command supports auditing and regression detection.

3.12.7 Three Pillars: Replayable → Queryable → Provable

The v2.2 Semantic Ledger Runtime achieves three mutually reinforcing pillars:

Pillar	Capability	Key Functions	CLI Commands
Replayable Queryable Provable	Rebuild state from Ledge Query Ledger as database Tamper-evident integrity	r rebuildState, validateTransition findProducer, findConsumer, ... hashLedger, ruleHash, diffLedgers	-validate, -ledger -query, -query-all, -stats -diff-ledgers

4. Semantic Validity Levels (SVL)

SVL is a formal taxonomy of AI-generated code correctness, providing layered, quantifiable verification guarantees:

Level	Name	Description	Guarantee
SVL-1	Symbol	Every called function, variable, and import	No hallucinated APIs.
SVL-2	Existence Type	exists in the project Argument count and	No type mismatches. No
SVL-3	Validity Dataflow	types match declared signatures Variables are	NameError /
SVL-4	Correctness	declared before use; no circular references	UnboundLocalError. No
SVL-5	Protocol Legality	Function call sequences respect declared	illegal state jumps.
SVL-5	Semantic Intent	state transitions Generated code faithfully	Aspirational; not
(Future)	Correctness	implements intended business logic	currently claimed.

Progmune Runtime v2.2 fully guarantees SVL-1 through SVL-4. v2.2 introduces Semantic Ledger invariant verification at the SVL-4 level, providing Invariant-0 + Invariant-1 dual consistency checks for every state transition. SVL-5 remains an open research challenge.

5. Experimental Evaluation

5.1 Stress Testing

Evaluated on synthetic TypeScript projects ranging from 3 to 338 functions. JSON structured output achieved 100% format correctness for the LLM Planner (compared to ~33% with the v1.0 DSL model). Constraint verification performance remained linear with respect to project size.

5.2 Semantic Blocking Tests

A test suite of 10 semantic intent cases demonstrated 80-100% blocking rate for semantic errors, with correct code generated in 7-8 out of 10 cases.

5.3 Multi-Namespace SSG Isolation

A protocol rule set of 18 rules spanning auth, file, and db namespaces was constructed. The SSG validator correctly intercepted cross-namespace state violations, providing namespace-attributed diagnostics and precise repair paths.

5.4 Antibody Immune Acceleration

ACL-4 antibody fast-path skips LLM calls entirely when matching globally stable antibodies, saving 100% of token consumption. ACL-3 antibodies guide LLMs toward known correct paths through prompt injection.

5.5 SSG Deterministic Repair

When protocol violations have known repair paths, `attemptSSGRepair()` automatically inserts missing functions without LLM retries. In the `dev_pipeline` workflow, when the LLM skips `validateAction` and attempts `emitCode` directly, the system auto-inserts the `validateAction` → `validateActionSequence` repair chain.

5.6 Semantic Ledger Invariant Verification ★v2.2

Complete invariant checks were performed on 6 real execution session Ledgers. All 6 Ledgers (11 transitions total: 7 valid, 4 invalid) passed triple verification of Invariant-0 + Invariant-1 + Replay. The integrity fingerprint `2f701ec86643dc63` covers all verified ledgers.

Invariant-1 negative test: a hand-crafted internally inconsistent transition was correctly flagged with 1 delta-consistency violation. This proves the invariant system detects internal data corruption.

5.7 Pure Function Architecture Verification ★v2.2

`p0_ssg_demo.ts` covers 7 end-to-end scenarios, all using pure functions (zero `StateMachineValidator` instances required):

- Scenarios 1-3: Valid/invalid auth sequence validation
- Scenario 4: Complete Pure Ledger trace

- Scenario 5: file/db namespace isolation
 - Scenario 6: Semantic Ledger pure function API + invariant negative test
 - Scenario 7: Ledger Query API (5 queries + integrity fingerprint)
-

6. Non-Goals

Progmune Runtime explicitly does not guarantee:

- Business logic correctness (e.g., whether a pricing calculation is accurate)
- Algorithm optimality or complexity
- Immunity to all security vulnerabilities (e.g., injection attacks, permission bypasses)
- Functional correctness of the entire application beyond the generated code unit

The system guarantees only program validity as defined by SVL-1 through SVL-4. Progmune is a program validity runtime, not a business correctness prover.

7. Future Directions

- **Global Immune Network:** Federated aggregation of anonymized Failure Corpus data across installations, enabling population-level immunity
- **Semantic Failure Benchmark:** The world's first public benchmark for AI-generated code reliability, built from aggregated, anonymized failure patterns
- **Enterprise Semantic Firewall:** Integration into CI/CD pipelines as a pre-merge gate for AI-generated pull requests
- **Deterministic Verifier (Rust/WASM):** Bit-identical verification across IDE, CI, and production environments
- **Observatory Web UI Enhancements:** Interactive state transition animation replay, antibody efficacy trend charts, cross-project immune network comparison. ★v2.2: Ledger timeline visualization, invariant violation highlighting, cross-session query console

- **Distributed Ledger Verification ★v2.2**: Publishing Ledger integrity fingerprints to immutable storage (e.g., content-addressable storage), enabling third-party independent verification of execution history
 - **Invariant-Driven Self-Healing ★v2.2**: When Invariant-1 detects an internally inconsistent transition, the system auto-corrects statesAfter to match acquired/invalidated — graduating from “detection” to “automatic repair”
-

8. Conclusion

Progmune Runtime v2.2 proves three things:

First, by establishing IR as the first principle, formalizing Runtime Ontology as serializable semantic records, and constraining LLMs as multi-layer heuristic proposers, we can achieve verifiable code synthesis with strong semantic guarantees.

Second, through the Semantic Ledger Runtime (★v2.2), the execution architecture of program immunity has been upgraded from Event Logging to Event Sourcing — the Transition Ledger is the system’s Single Source of Truth, with state as its derived view. The pure function architecture eliminates mutable state dependencies; dual invariants provide verifiable consistency guarantees for every transition; constraint snapshots and integrity fingerprints enable cross-version, tamper-evident deterministic verification.

Third, the three pillars — **Replayable** → **Queryable** → **Provable** — mark the maturity of the Runtime Kernel. The code immune system now has an auditable, verifiable, evolvable engineering foundation.

The layered SVL taxonomy, multi-namespace SSG protocol engine, ACL antibody inference system, SSG deterministic repair mechanism, accumulating Failure Corpus, three-tier memory architecture, and the Semantic Ledger Runtime’s pure-function invariant system together form a new kind of programming infrastructure: a neural-symbolic compiler runtime that learns, remembers, defends, and is now **auditable**.

We call this Program Immunology.

The system is available as open-source at <https://github.com/shenlian19831109/progmune-runtime> and as an npm package: `npm install progmune-runtime`.